1. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она илиможно внести некоторые доработки. Вы обратили внимание на таблицу с информацией о зарегистрировавшихся покупателях (клиентах). Таблица имеет следующую схему:

clients (

client_id number, --уникальный id клиента

client_name varchar(255), --имя клиента

client_surname varchar(255), --фамилия клиента

login varchar(30), --логин, который придумал клиент

city_id number, --id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)

age number, --возраст клиента

reg_date date –дата регистрации на сайте);

Считаете ли вы данный набор и смысл полей корректным? Если нет, то напишите что по вашему мнению некорректно и какие изменения внесли бы.

ОТВЕТ

Данный набор полей в таблице clients не является корректным, так как таблица денормализована. В текущем виде она не соответствует требованиям нормализации, что может привести к дублированию данных и затруднениям при обновлении информации. Для улучшения архитектуры и обеспечения структурированного хранения данных, я бы предложил привести схему к третьей нормальной форме (3NF), что является стандартом для построения корпоративного хранилища данных.

Пример:
1. Таблица клиентов (clients)

Хранит только информацию, относящуюся напрямую к клиенту, и не содержит дублирующих данных.

```
CREATE TABLE clients (
    client_id NUMBER PRIMARY KEY,       -- Уникальный ID клиента
    login VARCHAR(100) UNIQUE,          -- Уникальный логин клиента
    password_hash VARCHAR(255),         -- Хэш пароля клиента
    salt VARCHAR(255),                  -- Соль для хеширования пароля
    reg_date TIMESTAMP                  -- Дата регистрации на сайте
);
```

2. Таблица client_info (информация о клиенте):

CREATE TABLE client_info (

```
    client_id NUMBER PRIMARY KEY,       -- Уникальный ID клиента
    client_name VARCHAR(255),           -- Имя клиента
    client_surname VARCHAR(255),        -- Фамилия клиента
    birth_date DATE,                    -- Дата рождения клиента (вместо возраста, это добавляет
возможность, например, делать скидки клиентам ко дню рождения клиентов магазина, также не
будет такой проблемы, как обновление базы для актуализации возраста клиентов)
    CONSTRAINT fk_client FOREIGN KEY (client_id) REFERENCES clients(client_id)
);
```

3. Таблица городов (cities)

Хранит уникальные города с их ID и названиями.

```
CREATE TABLE cities (
    city_id NUMBER PRIMARY KEY,     -- Уникальный ID города
    city_name VARCHAR(255) NOT NULL -- Название города
);
```

4. Связь клиента и города (client_city)

Промежуточная таблица, связывающая клиентов с городами. Это нужно для соблюдения 3NF, где каждый атрибут в таблице клиентов должен зависеть только от первичного ключа.

```
CREATE TABLE client_city (
    client_id NUMBER,               -- ID клиента
    city_id NUMBER,                 -- ID города
    CONSTRAINT fk_client FOREIGN KEY (client_id) REFERENCES clients(client_id),
    CONSTRAINT fk_city FOREIGN KEY (city_id) REFERENCES cities(city_id)
);
```

5. Дополнительно можно создать таблицы для email и телефона

Это полезно, если у одного клиента может быть несколько email-адресов или телефонов (многие-ко-многим).

Таблица emails:

```
CREATE TABLE emails (
    email_id NUMBER PRIMARY KEY,    -- Уникальный ID email
    client_id NUMBER,               -- ID клиента
    email VARCHAR(255),             -- Email клиента
    CONSTRAINT fk_client_email FOREIGN KEY (client_id) REFERENCES clients(client_id)
);
```

6. Таблица phones:

```
CREATE TABLE phones (
   phone_id NUMBER PRIMARY KEY,    -- Уникальный ID телефона
   client_id NUMBER,               -- ID клиента
   phone VARCHAR(15),              -- Телефон клиента
   CONSTRAINT fk_client_phone FOREIGN KEY (client_id) REFERENCES clients(client_id)
);
```

Связи:

Таблица clients связана с таблицей client_info через внешний ключ client_id

Таблица clients связана с таблицей client_city через внешний ключ client_id

Таблица cities связана с таблицей client_city через внешний ключ city_id

Таблицы emails и phones связаны с таблицей clients через внешний ключ client_id

2. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она илиможно внести некоторые доработки. Вы обратили внимание на таблицу с информацией о товарах магазина.

Таблица имеет следующую схему:

```
 items (
item_id number, --уникальный id товара
item_name varchar(255), --наименование товара
item_cost number –стоимость товара
);
```

Глядя на эту таблицу вы вспоминаете, что мы живём в рыночной экономике и цена, если не каждый день, то каждый месяц может меняться. К примеру, сезонные повышения цен или наоборот скидки (перед началом учебного года или новым годом). Так же вы знаете что у магазина есть отдел финансовой отчётности, в котором сотрудникам нужно строить различныt отчёты (о доходах например) за разные промежутки времени, в том числе и за прошлые месяцы и даже прошлые года. Как бы вы доработали архитектуру таблицы, чтобы обеспечить историческое хранениестоимости товара?

ОТВЕТ

Я бы не смог ограничиться одной таблицей, так как корпоративное хранилище данных требует нормализации для минимизации дублирования и поддержания целостности данных. Для эффективного хранения и анализа информации необходимо разделить данные на логически независимые таблицы. Это позволит не только избежать избыточности, но и обеспечит гибкость при расширении и модификации базы, позволяя корректно хранить исторические данные и легко интегрировать новые функции, такие как отчетность.

Пример:
1. Таблица items (основная информация о товарах):

Эта таблица будет хранить информацию о самих товарах.

```
CREATE TABLE items (
   item_id NUMBER PRIMARY KEY,       -- Уникальный ID товара
   item_name VARCHAR(255) NOT NULL   -- Наименование товара
);
```

2. Таблица item_prices (история изменения цен):

Для того чтобы учитывать историю изменения цен на товары, создаем таблицу с историей цен.

```
CREATE TABLE item_prices (
   price_id NUMBER PRIMARY KEY,      -- Уникальный ID записи о цене
   item_id NUMBER NOT NULL,          -- ID товара (связь с таблицей items)
   item_cost NUMBER NOT NULL,        -- Стоимость товара
   start_date DATE NOT NULL,         -- Дата начала действия цены
   end_date DATE,                    -- Дата окончания действия цены (NULL, если цена актуальна)
   CONSTRAINT fk_item FOREIGN KEY (item_id) REFERENCES items(item_id)
);
```

3. Таблица promotions (информация о скидках):

```
CREATE TABLE promotions (
   promo_id NUMBER PRIMARY KEY,         -- Уникальный ID промо-акции
   promotion_name VARCHAR(255) NOT NULL -- Название или описание промо-акции
);
```

4. Таблица promotion_items (история скидок на товары):

```
CREATE TABLE promotion_items (
   promo_item_id NUMBER PRIMARY KEY,     -- Уникальный ID записи
   promo_id NUMBER NOT NULL,             -- ID промо-акции (связь с таблицей promotions)
   item_id NUMBER NOT NULL,              -- ID товара (связь с таблицей items)
   discount_percent NUMBER NOT NULL,     -- Процент скидки
   start_date DATE NOT NULL,             -- Дата начала действия скидки
   end_date DATE,                        -- Дата окончания действия скидки
   CONSTRAINT fk_promo FOREIGN KEY (promo_id) REFERENCES promotions(promo_id),
   CONSTRAINT fk_item FOREIGN KEY (item_id) REFERENCES items(item_id)
);
```

Таблицы:

Таблица items хранит информацию о товаре.

Таблица item_prices хранит историю цен на товар.

Таблица promotions хранит информацию о промо-акциях (общая информация, например, название или тип акции).

Таблица promotion_items хранит историю применения скидок к товарам, связывая товары с промо-акциями.

3. Требуется проверить запрос на корректность и исправить там ошибки, если они есть. Существует учебная схема HR, содержащая таблицы: employees, departments и locations Необходимо получить все отделы0, расположенные в Сеуле в которых все сотрудники, не имеющие менеджера, зарабатывают в общей сложности более 100000. Для решения задачи был написан следующий запрос:

SELECT DEPARTMENT_ID,

DEPARTMENT_NAME,

SUM(SALARY) TOTAL_SALARY

FROM EMPLOYEE E, DEPARTMENTS D

WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID AND MANAGER_ID = NULL AND LOCATION_ID = (SELECT LOCATION_ID FROM LOCATIONS WHERE CITY = 'SEOUL') AND SUM(SALARY) >= 100000 GROUP BY DEPARTMENT_NAME

ОТВЕТ

1. Ошибка в группировке, не учитывается поле DEPARTMENT_ID

2. WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID AND MANAGER_ID = NULL AND LOCATION_ID = (SELECT LOCATION_ID FROM LOCATIONS WHERE CITY = 'SEOUL') AND SUM(SALARY) >= 100000 – вот это вы забавно придумали))))
Не хватает джоинов, фильтр по зарплате надо перенести в HAVING, тк SUM() – агрегатная функция, подзапрос для фильтрации города убираем, также по формулировке (зарабатывают в общей сложности более 100000), наверное лучше использовать знак >.

3. FROM EMPLOYEE E, DEPARTMENTS D очевидно ошибка

Пример исправленного запроса:

```
SELECT

   D.DEPARTMENT_ID

   ,D.DEPARTMENT_NAME

   ,SUM(E.SALARY) AS TOTAL_SALARY

FROM

   EMPLOYEES E

INNER JOIN DEPARTMENTS D

            ON E.DEPARTMENT_ID = D.DEPARTMENT_ID

INNER JOIN LOCATIONS L
```

```
        ON D.LOCATION_ID = L.LOCATION_ID
WHERE
  E.MANAGER_ID IS NULL
  AND L.CITY = 'SEOUL'
GROUP BY
  D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING
  SUM(E.SALARY) > 100000;
```

4. В некоторой комнате на пол уронили карандаш. Объясните почему вы не можете через него перепрыгнуть?

ОТВЕТ

Здесь можно придумать много решений.

1. Самое очевидное – зачем в принципе прыгать через карандаш? Его можно просто поднять или взять другой карандаш, если некогда лазать по полу в поисках карандаша.

2. Карандаш мог закатиться под мебель, например, под стол – через него нельзя перепрыгнуть. Ровно также, если он укатился в угол комнаты или к стене, если комната объята пламенем или затоплена водой.

3. В некоторой комнате уронили карандаш, в некоторой, это какой и где конкретно, я в этой комнате нахожусь? Если нет, то я не могу через него перепрыгнуть.

4. И все остальные истории с уточняющими вопросами, поскольку без них можно думать бесконечно много.